

DEPTH-ADAPTIVE INFERENCE OF LOOPED LANGUAGE MODELS VIA CONTINUOUS DEPTH BATCHING

Kristian Schwethelm¹ Daniel Rückert^{1,2,3} Georgios Kaissis⁴

¹Chair for AI in Healthcare and Medicine, Technical University of Munich (TUM), Germany

²Department of Computing, Imperial College London, UK

³Munich Center for Machine Learning (MCML), Germany

⁴Hasso Plattner Institute for Digital Engineering, University of Potsdam, Germany

ABSTRACT

A main promise of looped language models (LMs) is depth-adaptive inference. By iterating a block of shared layers a variable number of times, the model can use less compute for “easy” tokens and more for “hard” ones. However, this adaptivity breaks standard batching: tokens in the same batch now require a different number of loops, so there is no unified forward pass, making efficient inference difficult. Standard inference frameworks like vLLM schedule on the token level and cannot handle this because tokens need to be removed from the batch *within* the forward pass. Loop-level scheduling has been proposed as a solution, but never implemented end to end. The key challenge is that looped architectures also contain non-looped boundary stages (e.g., token embedding and LM head) that must be scheduled at different frequencies than the loop. We introduce *continuous depth batching* (CDB), which schedules at the granularity of individual loop iterations. CDB handles boundary stages and loop steps in separate priority queues, makes exit decisions one step ahead, and overlaps all scheduling work with GPU computation. On Ouro 1.4B and Huginn 3.5B, CDB can realize up to 99% of the theoretical maximum speed-up from adaptive-depth, translating to 1.5–1.9× higher offline throughput and 45–90% lower normalized latency under dynamic serving load.

1 INTRODUCTION

Processing sequences in parallel has become central to many large language model (LLM) applications. For example, when users interact with an online API, multiple requests must be served simultaneously by an LLM service such as Claude or ChatGPT. However, online serving is not the only setting where multiple sequences must be handled at once. Post-training LLMs with reinforcement learning requires generating many rollouts, benchmarking requires running many evaluation samples, and locally hosted multi-agent coding assistants make concurrent LLM calls. At the same time, reasoning models can generate thousands of tokens for a single request, substantially increasing the required inference compute (Guo et al., 2025). Efficient batching, i.e., serving many sequences in a single forward pass, is therefore critical to making LLMs practical to train, deploy, and use.

At the hardware level, a major bottleneck is autoregressive decoding, where LLMs generate responses one token at a time. Without batching, the model weights must be loaded from memory to operate on a single token vector, which takes much more time than performing the actual arithmetic; the forward pass is *memory bound*. Larger batches combine next-token predictions from multiple sequences into a single forward pass, reusing the loaded weights and thereby improving hardware utilization.

LLM architectures and inference systems are increasingly co-designed (Anthony et al., 2024; Bian et al., 2026), but looped language models have received little attention from the inference perspective. A looped LM iterates a block of shared layers, called the *recurrent core*, to refine the latent representation before predicting a token. Layers outside the core, such as the token embedding, LM head, or un-shared transformer layers, run only once per token; we refer to these boundary stages as the *prelude* and *coda* (Geiping et al., 2025). When the number of iterations is fixed, the architecture can be unrolled into a standard transformer with shared weights, and existing serving frameworks like vLLM (Kwon et al., 2023) support it with minimal modification. However, a key advantage of looped

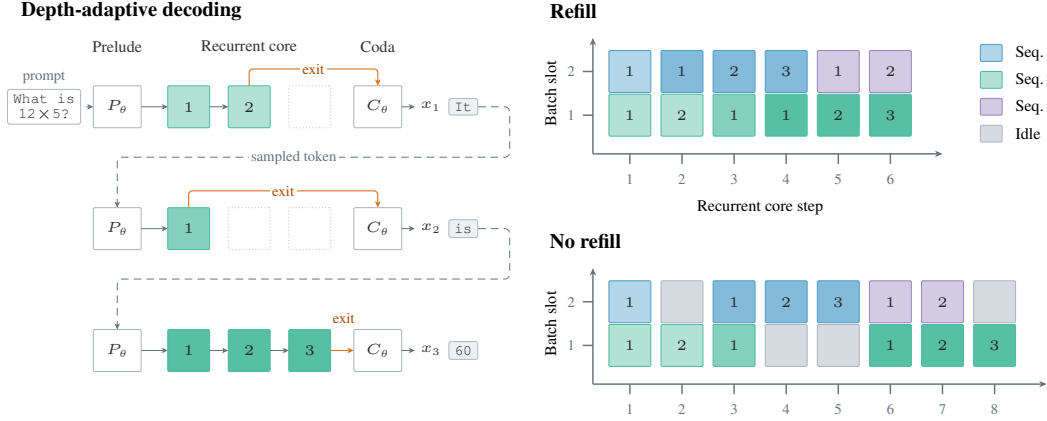


Figure 1: *Left*: looped LM decoding three tokens x_t , each using a different number of loops r_t . Every decode step runs the prelude P_θ once, loops the recurrent core up to $r_{\max} = 3$ times, and finally runs the coda C_θ , which samples the next token. *Right*: three requests under CDB with two batch slots and depth-adaptive looping. Cell color identifies the sequence and darker tints mark later tokens. Refill fills slots freed by an exit with a new token, keeping a larger batch size and finishing in 6 forward passes. No refill instead reduces the batch size on token exits, needing 8 passes for the same work.

LMs is *depth-adaptive* decoding, where each token can exit the loop after a different number of steps (see Fig. 1, left). “Easy” tokens loop less and thus take less compute than “hard” ones, yielding a better compute–accuracy trade-off (Geiping et al., 2025; Zhu et al., 2025). But this adaptivity breaks the uniform forward pass of a batch, as tokens now follow different paths through the model. The boundary stages add further complexity because when a token exits the core, the coda must run for that token alone while other tokens continue looping. The inference scheduler must therefore decide not only when tokens enter and leave the batch but also when each stage runs and for which tokens. So far, no inference system supports this. In this paper, we ask: *Can depth-adaptive looped LMs be served efficiently, and if so, how? How does the architecture affect inference efficiency?*

We introduce *continuous depth batching* (CDB), an inference method that schedules at the granularity of individual loop iterations, routing tokens between stages *within* the forward pass. We build CDB in two modes (Fig. 1, right). *No-refill* only lets the recurrent batch shrink, so exited tokens wait until the entire batch finishes before running the coda together. *Refill* maximizes throughput by scheduling stages independently via a priority queue system, which interleaves boundary stages with recurrent steps and fills freed recurrent batch slots with new tokens (Bae et al., 2025a). Both modes require new KV-cache layouts, as tokens at different exit depths leave gaps in the standard depth-indexed cache. Finally, to minimize per-step scheduling overhead, we overlap batch preparation with GPU execution by introducing a lookahead gate that makes exit decisions one loop step in advance.

We analyze the maximum speed-up from depth-adaptive inference by deriving a FLOPs-based bound. We also fit a hardware-aware roofline model that identifies the batch-size regimes where the refill mode is most beneficial. On Ouro 1.4B (Zhu et al., 2025) and Huginn 3.5B (Geiping et al., 2025), CDB realizes up to 99% of the theoretical maximum speed-up, which translates to 1.5–1.9 $\times$ higher offline throughput and 45–90% lower normalized latency under online serving.

Contributions.

1. We implement continuous depth batching (CDB) end to end for depth-adaptive looped LMs, with a high-throughput refill and a simpler no-refill execution mode.
2. We derive an upper bound on the speed-up from adaptive depth and identify the batching regime in which refill is beneficial.
3. We evaluate CDB on two looped LM architectures and show how their structure, the loop count and the cost of boundary stages, enables efficient inference.

2 PROBLEM SETUP

Looped LMs are decoder-only transformers whose layers are split into a non-recurrent *prelude* P_θ , a shared *recurrent core* R_θ , and a non-recurrent *coda* C_θ . The recurrent core can be looped a variable number of times, while prelude and coda are run once. For example, a model with a 2-layer prelude, a 4-layer core, and a 2-layer coda executes $2 + 4r_t + 2$ layers for a token that takes r_t loops. At $r_t = 4$ this is an effective depth of 20 layers. A fully looped model has no transformer layers outside the core, just the token embedding and the LM head, and reaches the same effective depth by looping the 4-layer core five times. We denote the layer split as prelude-core-coda, e.g., 2–4–2 and 0–4–0. For token position t , let $h_t^{(r)}$ denote the latent state after r loops, and let $\mathcal{K}_{<t}^{(r)}$ denote the key-value (KV) states of earlier positions for attention computation. A forward pass for token x_t can be written as:

$$h_t^{(0)} = P_\theta(x_t), \quad h_t^{(r+1)} = R_\theta\left(h_t^{(r)}, \mathcal{K}_{<t}^{(r)}\right), \quad p(x_{t+1} \mid x_{\leq t}) = \text{softmax}\left(C_\theta(h_t^{(r_t \leq r_{\max})})\right), \quad (1)$$

where θ denotes the model parameters, $r_{\max}$ the maximum loops, and r_t the loops actually used to predict token x_{t+1} . Depth-adaptive decoding lets r_t vary per token: after each loop the runtime compares an exit signal against a threshold and stops once the exit criterion is satisfied. Crucially, each token leaves the core at an *input-independent* depth, decided *at the end of a loop step*.

This per-token variability creates a problem for batched inference. Standard *continuous batching* (CB) schedules only at token boundaries: each tick, active sequences generate one token, completed sequences are removed, and new tokens are added to the active batch (Yu et al., 2022). Since CB treats each forward pass as a single scheduling unit, a looped LM can only run every token in a batch for the same, predefined number of loops. But when tokens exit the recurrent core at different times, one after two loops and another after four, the batch is no longer uniform. The exited token needs to run the coda to produce its next-token prediction, but the remaining tokens need another recurrent step, and there is no mechanism in CB to run different operations for different tokens within a forward pass. To support this, the scheduler must operate *within* the forward pass by tracking each token’s loop progress individually and routing tokens between stages as they exit. We call this *continuous depth batching* (CDB). A successful implementation must address three main challenges: (1) *stage heterogeneity*—a CB step is one uniform forward pass, while CDB must schedule the prelude, recurrent core, and coda separately, each at different batch sizes and frequencies; (2) *critical path exits*—the exit decision is available only *after* step r but determines which tokens enter step $r + 1$, so the GPU idles after each loop step until the scheduler has processed the exit; (3) *ragged KV*—a token exiting at r_t writes no KV states for deeper steps, so the KV cache must be depth-aware.

3 RELATED WORK

Depth-adaptive looped transformers. Existing methods control per-token depth in two ways. Early-exit gates decide for each token individually when it leaves the loop. Learned variants attach a small head to the recurrent core (Banino et al., 2021; Byra et al., 2026; Frey et al., 2026; Fu et al., 2025; Li et al., 2026; Song et al., 2026; Tan et al., 2023; Zhu et al., 2025), while training-free variants compare hidden states or output logits between consecutive steps and exit once they converge (Geiping et al., 2025; Kohli et al., 2026; Lee et al., 2026; Shu et al., 2026; Tur et al., 2026). Mixture-of-Depths-style works instead route a fixed-size subset of tokens through each loop step, so a token’s depth depends explicitly on the other tokens in the sequence (Bae et al., 2025b; Mohtashami et al., 2025; Raposo et al., 2024). In contrast to these works, we study per-token adaptive depth from the *serving* side. CDB executes a given exit rule without changing the model’s predictions, so it is an implementation technique compatible with the methods above.

Serving looped LMs. Continuous depth batching was introduced by Bae et al. (2025a) as the idea of refilling depth slots freed by early exits with new samples, and adopted in (Bae et al., 2025b). Neither work implements it end-to-end leaving the challenges of Section 2 unaddressed. The former work relies on oracle exit traces and hypothetical generation-speed estimates, while the latter omits prefill, scheduling, and KV-cache updates. We provide the first end-to-end implementation of CDB, including a serving scheduler, exit gate, and KV-cache handling, and evaluate it on real workloads.

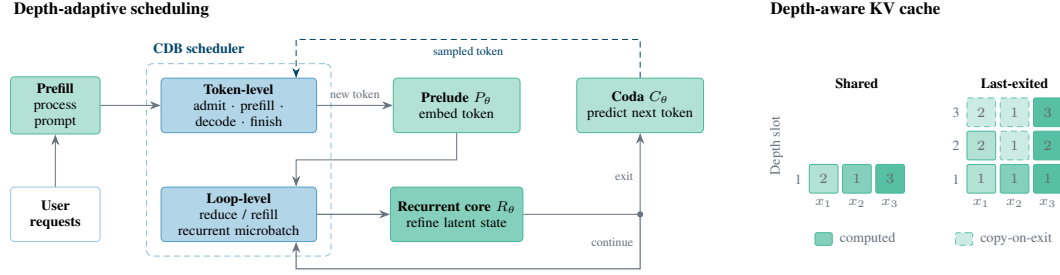


Figure 2: *Left*: we decompose generation into four queues: prefill for waiting requests, and prelude, recurrent core, and coda for decode. After each loop step, the exit decision routes tokens either back to the recurrent queue or out to the coda. *Right*: two depth-aware KV-cache designs for three tokens that exit at depths $(r_1, r_2, r_3) = (2, 1, 3)$. Each cell shows the loop step whose state is stored.

4 METHOD: CONTINUOUS DEPTH BATCHING

We now describe how our CDB method handles the three challenges described in Section 2. We address stage heterogeneity through a queue-based decode scheduler, ragged KV through two depth-aware KV cache layouts, and critical-path exits through asynchronous execution.

4.1 QUEUE-BASED DECODE SCHEDULER

We split each request’s path from prompt to completed response into four queues: inactive requests waiting for prefill, decode tokens awaiting the prelude, recurrent items ready for their next loop step, and exited tokens awaiting the coda (Fig. 2, left). The three decode queues correspond to the three stages of the looped architecture, which the scheduler runs as separate operations. Each tick, the scheduler selects one queue, batches the tokens waiting in it, and processes them through that stage. Because the stages are decoupled, their execution order is flexible. For example, the scheduler can run two consecutive coda batches for different tokens before starting the next recurrent step.

Based on this queuing system, we implement two execution modes that differ in how stages are ordered. In *no-refill* mode, the decode stages run in a fixed order: prelude, then repeated core steps until all tokens have exited, then coda. Exited tokens are removed from the recurrent batch, reducing compute on subsequent steps, but no new tokens enter until the next decode round. In *refill* mode, the stages are free to run in any order, so if a token exits the loop its freed slot can directly be filled by another token, keeping batch size high. This is possible because the recurrent core reuses the same weights at every loop step: a token on its first pass through the core and a token on its fourth share the same layers, so they can run together in the same batch. For example, in Fig. 1 (right), token x_2 of Seq. 1 refills the batch with its first loop step while Seq. 2’s first token is still running its second.

When the stages can run in any order, the scheduler needs a priority rule. We let the prelude always run directly after prefill or coda, as there is no routing decision between these stages. The remaining three queues involve actual decisions: new requests arrive at the prefill queue, and the exit decision routes tokens between the recurrent and coda queues. Among these, we give the coda highest priority, since it finishes requests, then prefill up to a KV-cache limit, since admitting prompts early allows for larger batches during decode. However, when the coda contains many transformer layers, running it after every single loop exit is costly. Even when one token exits the loop and 255 remain, the single-token coda has priority and runs before the 255 can continue looping. In such cases, we set a minimum batch size for the coda, so enough exits accumulate. The trade-off is that tokens now take longer to reach the recurrent queue, so fewer tokens are available for refill.

4.2 DEPTH-AWARE KV CACHE

A looped LM writes KV states for each loop step, so shared layers get $r_{\max}$ KV cache entries. When a token exits early ($r_t < r_{\max}$), the slots for its remaining steps are never written, so a later token looping deeper finds empty entries. We implement a *last-exited cache* that fills these empty slots by copying the last state into the slots of the skipped steps (Fig. 2, right) (Geiping et al., 2025; Zhu

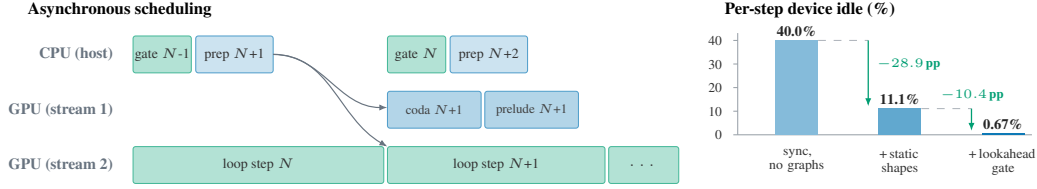


Figure 3: *Left*: the CPU prepares the next batch ($N+1$) from the previous step’s exit decision ($N-1$) while the GPU runs the current loop step in parallel (N). We also overlap the prelude and coda on a second GPU stream. *Right*: without CUDA graphs, the GPU idles 40% of the time. Static shapes reduce idle time by 28.9 percentage points, and the lookahead gate removes a further 10.4 percentage points, leaving only 0.67% idle time. Our CDB scheduler therefore incurs negligible overhead.

et al., 2025). However, this wastes memory and compute on copies. As a more memory-efficient alternative we implement a *shared cache*, which gives each layer only one KV slot, saving $r_{\max}$ times the memory of the last-exited cache. The shared KV slot gets overwritten at every loop step, refining the KV states instead of writing new ones. A later token then always attends to each earlier token’s most recent state, regardless of when it exited, which avoids the empty-slot problem entirely with no copying or routing of KV states. However, this changes the attention semantics (Geiping et al., 2025; Vendrell et al., 2026; Zhu et al., 2025). Both layouts are compatible with modern attention kernels like paged attention (Kwon et al., 2023) and FlashAttention-3 (Shah et al., 2024).

4.3 EFFICIENT ASYNCHRONOUS EXECUTION

The CDB scheduler runs on CPU, while the model runs on specialized accelerators like GPUs. Between GPU operations, the scheduler reads back results, selects the next queue, assembles a new batch, and copies the required data to the GPU. The GPU is briefly idle during this. Since we split the forward pass into separate stages, we have many scheduler interactions and the idle time adds up substantially. We hide the scheduler’s overhead by running it asynchronously, i.e., batch preparations and data copies are made in parallel while the GPU processes the current batch (Fig. 3, right). The coda and prelude also run in parallel on separate GPU streams, which improves GPU utilization when batch sizes of the stages are small. For example, when a single token exits the loop, its coda stage can run in parallel with the next loop step of the remaining tokens. We also use acceleration techniques like static, preallocated tensors, CUDA graphs, and batches padded to the next power of two.

However, there is a problem with preparing the next batch in advance: the gate output at loop step r determines which tokens leave the batch, but is only available once step r finishes on the GPU. The CPU must read back the result and build the batch before the GPU can start step $r+1$. End-of-sequence (EOS) tokens pose an analogous problem on the token level, which is usually handled by assuming no EOS and catching it one step late, causing one extra generated token. We propose a similar idea with a *lookahead gate*. The exit signal at step r determines whether a token participates in step $r+2$ instead of $r+1$, giving the CPU a full loop step to prepare the next batch. Unlike the EOS approach, this does not waste compute, since we decide the exit earlier instead of catching it later. In Appendix C.2, we show that the exit decision can be made one step earlier without reducing accuracy. Overall, our CDB implementation reduces GPU idle time to nearly zero (Fig. 3, right).

5 EFFICIENCY LIMITS OF ADAPTIVE DEPTH

Each early exit saves recurrent-core FLOPs, but fewer FLOPs do not necessarily reduce hardware wall-clock time, and larger batches are not automatically faster either. In this section, we first bound the achievable FLOP savings, then model the latency of a recurrent step to determine when larger batches reduce wall-clock time. We focus on decode, which usually dominates inference time. As examples, we take the looped models Ouro (Zhu et al., 2025) and Huginn (Geiping et al., 2025).

FLOP savings. Let F_r be the FLOPs of one recurrent-core application for a single token, and F_0 the loop-independent FLOPs of the prelude and coda. Fixed-depth decoding requires $F_0 + r_{\max} F_r$ and adaptive-depth decoding $F_0 + r_t F_r$. For T generated tokens with mean exit depth $\bar{d} = \frac{1}{T} \sum_{t=1}^T r_t$,

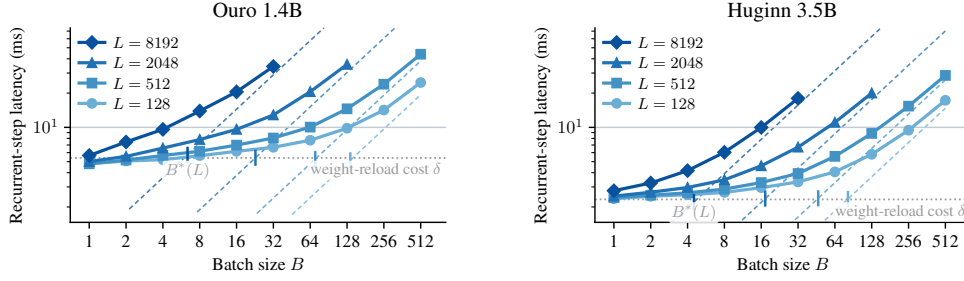


Figure 4: Decode latency of a recurrent step versus batch size at context lengths L , for Ouro (left) and Huginn (right). The dotted line shows the weight-reload cost δ , with blue ticks marking the saturation batch size $B^*(L)$. The dashed line shows compute-bound scaling, where latency grows linearly.

the achievable speedup from early-exit is at most the *FLOP bound*

$$\frac{F_0 + r_{\max} F_r}{F_0 + \bar{d} F_r} \leq \frac{r_{\max}}{\bar{d}} \leq \frac{r_{\max}}{r_{\min}}. \quad (2)$$

For Ouro, where prelude and coda are only the token embedding and LM head ($F_0 \ll F_r$) the bound is close to $r_{\max}/\bar{d}$. With $r_{\max} = 4$, each step less saves about 25% of per-token FLOPs. For Huginn’s 2-4-2 layer split, the prelude and coda cost about as much as the recurrent core ($F_0 \approx 1.2 F_r$), so the bound falls below $r_{\max}/\bar{d}$. Additionally, with $r_{\max} = 16$, one step less saves only about 6%. Note that the bound counts only decode FLOPs and ignores prefill, scheduling, and KV cache overhead.

Recurrent-step latency. A recurrent step reloads the core’s weights, runs the matrix multiplications, and loads the KV cache for attention layers. The latency of these components scale differently: the weight reload is a fixed cost, while compute and attention latency grow with batch and context size. With N_b batch tokens and N_c context tokens (KV entries), we estimate the total recurrent step latency with a roofline model (Williams et al., 2009) as $t(N_c, N_b) = \delta + \gamma N_b + \alpha N_c$, where δ is the fixed weight-reload cost, γN_b the recurrent core cost, and αN_c the KV cache streaming cost.

In Fig. 4, we measure the latency of a single recurrent step of Ouro 1.4B and Huginn 3.5B for different batch sizes and context lengths, and fit our roofline model. Both LMs show a flat latency curve at small batch sizes. Here, the matrix multiplications finish quickly and most time is spent reloading the recurrent core’s weights. Since the weights must be reloaded regardless of how many tokens are in the batch, the step takes roughly the same time whether it processes 1 token or 8; the step is *memory bound*. As the batch grows, the matrix multiplications eventually take longer than the weight reload, and latency begins to scale linearly with batch size; the step becomes *compute bound*. The *saturation batch size* B^* marks the transition between the two regimes. B^* falls with context length L , because longer contexts increase the KV-cache streaming cost per token. See Appendix A for detailed results.

These compute regimes determine when CDB’s refill mode is most effective. In the memory-bound regime, early exits from a batch save FLOPs but not wall-clock time: the step takes δ regardless, so simply reducing the batch size as tokens exit (no refills) saves no time. The refill mode recovers this by filling freed slots with new tokens, using otherwise idle compute. In the compute-bound regime, wall-clock time scales with total work, so early exits reduce latency even without refill.

6 EXPERIMENTS

We evaluate CDB in two settings. *Offline throughput* measures how fast the engine processes a fixed workload where all requests are known upfront, as in benchmarking or RL, so the engine runs under sustained load. In addition to raw speed, this lets us test how close CDB comes to the FLOP bound of Eq. (2) and how well the decode-step roofline (Section 5) predicts different load scenarios. *Online serving* replays the same requests with dynamic arrivals, simulating user queries that arrive at different times, so that queueing effects and varying load impact efficiency. Task accuracy depends on the model served; we ablate KV cache sharing and exit gates in Appendix C.

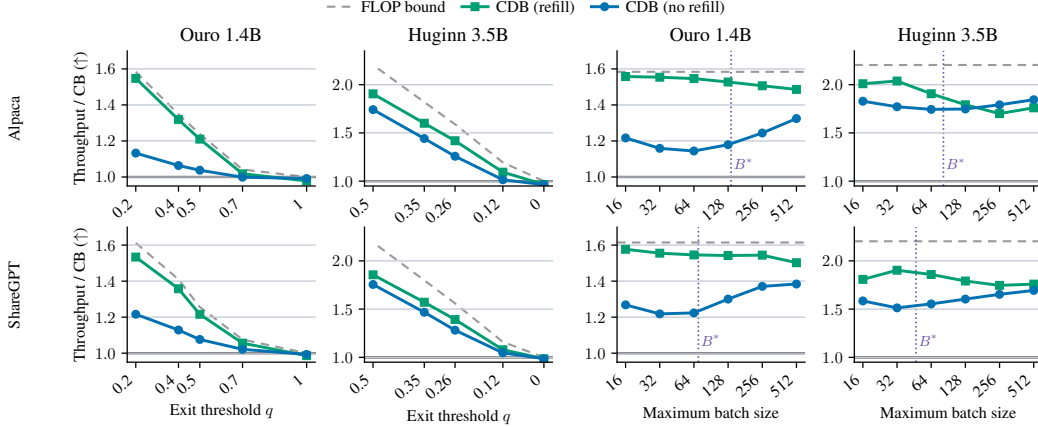


Figure 5: Offline throughput normalized to CB (full depth) for Alpaca (top) and ShareGPT (bottom). *Left*: exit threshold q sweep for maximum batch size of 64. *Right*: maximum batch size sweep at $q = 0.2$ (Ouro) and $q = 0.5$ (Huginn); dotted line is B^* from Section 5.

6.1 SETUP

We construct workloads from Alpaca (Taori et al., 2023) (short instructions) and ShareGPT (ShareGPT Team, 2023) (long chatbot requests), following Kwon et al. (2023). ShareGPT requests contain about $15\times$ more input and $5\times$ more output tokens than Alpaca. Each request’s output length is set to a reference response’s token count and exit decisions replayed from the LLM’s exit gates at multiple thresholds, so the depth distribution is realistic but reproducible across backends (Appendix B).

We evaluate two state-of-the-art looped LMs that represent the main architecture families. Ouro 1.4B (Zhu et al., 2025) is fully looped (0-24-0, $r_{\max} = 4$, 5.6B effective compute), so the boundary stages are cheap enough to run at every exit, overlapped with recurrent steps at negligible cost (Section 4.3). Huginn 3.5B (Geiping et al., 2025) contains transformer layers in prelude and coda (2-4-2, $r_{\max} = 16$, ~ 28 B effective compute). To reduce their cost, we set a minimum batch size of 32 for Huginn’s boundary stages. This means that tokens exiting the recurrent core are not immediately processed through the coda and prelude but wait until 32 exits have accumulated, which can delay re-entry into the recurrent core by several loop iterations. Both models use a shared KV cache.

We compare three inference engines. Standard CB, which schedules at the token-level and always runs $r_{\max}$ recursions; CDB (no refill), which removes exited tokens and lets the batch shrink; and CDB (refill), which fills freed slots with new work. All three share the same codebase (adapted from Hugging Face Transformers (Wolf et al., 2020) and vLLM (Kwon et al., 2023)), kernels, and admission policy. All experiments run on a single 80 GB H100 GPU.

6.2 OFFLINE THROUGHPUT

We submit all requests at once and report generated tokens per second, normalized to the CB baseline, which can only run at full depth. Fig. 5 (left) sweeps the exit threshold q at a maximum decode batch size of 64, near the saturation batch size B^* (Section 5). The right side sweeps the maximum decode batch size at an aggressive threshold with many early exits ($q = 0.2$ for Ouro, $q = 0.5$ for Huginn); other thresholds show the same pattern with a smaller spread between curves.

How close does CDB get to the FLOP bound? The FLOP bound (dashed line) is the theoretical maximum speed-up from early exit, excluding prefill and scheduling. The FLOP bound reaches up to $\sim 1.6\times$ for Ouro and $\sim 2.4\times$ for Huginn. For Ouro, CDB (refill) nearly maximizes the theoretical speed-up, reaching 94–99% of the bound on both workloads, with Alpaca being slightly higher because it has fewer input tokens and prefill is a smaller fraction of total time. Even without early exits, where no speed-up is possible, CDB stays within 2% of CB, showing that our scheduler has

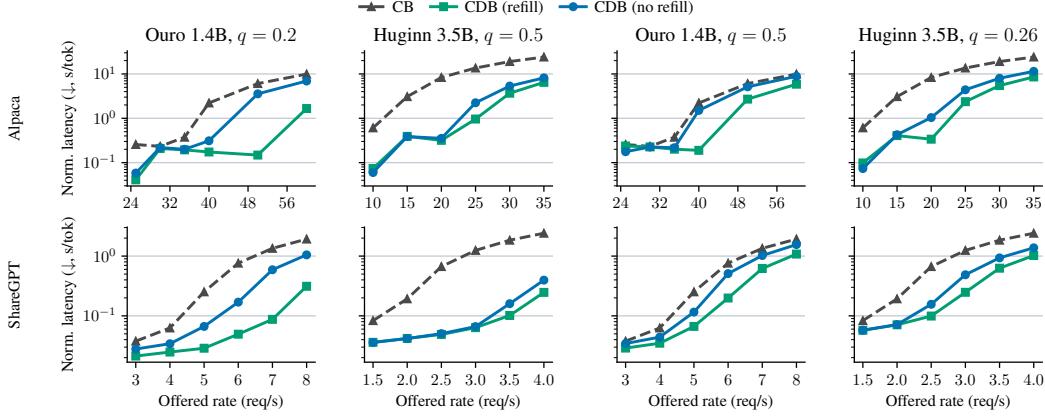


Figure 6: Normalized latency against request rate at two exit thresholds per model, on Alpaca (top) and ShareGPT (bottom). The two left columns contain thresholds with many exits, and the two right columns thresholds with less exits. CB is threshold-independent (full depth).

negligible overhead. For Huginn, CDB also provides substantial speed-up up to $1.9\times$ but is less efficient (73–92%) due to large boundary stages, which add much latency between loop steps.

When is refill most effective? For Ouro, CDB (refill) remains close to the FLOP bound across the full batch-size sweep in Fig. 5 right ($1.5\text{--}1.58\times$ speed-up). CDB (no refill) gets close at large batches ($\sim 1.4\times$) but drops to $\sim 1.2\times$ below B^* , where shrinking the batch saves less wall-clock time due to the memory-bound regime. Refill prevents this by filling freed slots with new work, confirming the cost model’s prediction that refill’s advantage is largest in the memory-bound regime (Section 5). For Huginn, the advantage of refill over no-refill is smaller due to the boundary stage latency. On Alpaca, refill even drops lower at large batch sizes. This is because larger batches produce more exits per loop step, so the minimum coda batch size of 32 is reached more often and the coda runs too frequently. The minimum coda batch size should thus scale with the decode batch size.

Overall, CDB delivers substantial speed-up for both architectures. For fully looped models like Ouro, CDB’s refill mode can maximize performance, nearly reaching the theoretical FLOP bound. For models with large boundary stages like Huginn, CDB still achieves up to $1.9\times$ speed-up.

6.3 ONLINE SERVING

We replay the same workloads with random arrivals (Poisson process) at average rate λ for a 10-minute window. We report normalized latency (end-to-end time divided by output length), which controls for varying response lengths across requests (Kwon et al., 2023; Yu et al., 2022). Fig. 6 shows the results for two gate thresholds per model.

At aggressive thresholds with many early exits ($q = 0.2$ for Ouro, $q = 0.5$ for Huginn), CDB (refill) delivers the lowest normalized latency across the full rate sweep. At moderate loads, CDB (refill) reduces normalized latency by 45–57% on ShareGPT and by 84–88% on Alpaca. The ordering matches the offline results: CB saturates first, CDB (no refill) next, and CDB (refill) last, leaving a band of rates that only refill can serve. At thresholds with fewer early exits the gap narrows and the three curves converge, as there is little to no time to save.

Whether refill helps at low load depends on the boundary-stage latency. For Huginn, we batch the boundary stages until 32 exits accumulate, so the exited token takes several loop iterations to pass through coda and prelude and return to the recurrent queue. At low load, no other tokens are waiting, so the freed slot stays empty until that token returns, and refill and no-refill produce nearly identical latency. For Ouro, the coda and prelude run immediately, so the exited token returns to the queue within one loop step and can refill the slot it left. This is more visible on ShareGPT because the longer outputs keep more tokens decoding at the same time. On Alpaca, outputs are shorter and fewer tokens are available to refill.

7 DISCUSSION

Co-designing looped LMs and inference systems. The model architecture has a large impact on inference efficiency. CDB substantially increases inference speed by refilling the batch when tokens exit early. However, refill needs tokens waiting in the recurrent queue and tokens only get there after running the boundary stages (prelude and coda). If those stages are costly, they cannot run as often, leaving fewer tokens ready for refill. This favors fewer non-looped layers, which is in tension with recent findings that adding layers outside the recurrent core improves accuracy under matched training compute (Bae et al., 2025b; Geiping et al., 2025). Similarly, compute-optimal models use fewer loops (Schwethelm et al., 2026), but more loops widen the range for adaptive depth. Early exits can reduce accuracy, so depth-adaptive decoding is a compute-accuracy trade-off controlled by the exit threshold. This is analogous to test-time compute scaling in reasoning models, where fewer thinking tokens reduce quality but save cost. The KV cache layout is also important as a shared cache substantially reduces the memory footprint and implementation complexity of CDB. Looped LMs are usually robust to shared KV cache, but some architectures may need finetuning (Appendix C.1).

Choosing the CDB mode. The no-refill mode is much simpler to implement because the batch only shrinks, so prelude, core, and coda stages run in a fixed order without sequence routing in between. Refill adds scheduling complexity requiring a priority and queuing system for different stages, which depending on the model and workload may need to be tuned carefully. While refill is mostly substantially faster, especially in the memory-bound regime, above B^* the two modes converge (Section 5), so no-refill may be preferable when simplicity matters more than peak throughput.

Limitations. The lookahead gate imposes a minimum depth of $r_{\min} = 2$, which reduces the available exit range and limits the achievable speed-up, especially at low $r_{\max}$. We only evaluate the shared KV cache; other layouts may behave differently and require a dedicated implementation.

Future work. Better exit gates and training objectives for adaptive depth would allow more aggressive thresholds at the same accuracy, directly translating to higher throughput through CDB. Chain-of-thought reasoning adds a second dimension of variable compute: not only per-token depth but also the number of generated tokens becomes adaptive. How the two interact is an open question. Combining CDB with other efficiency methods like multi-GPU parallelism, speculative decoding, and chunked prefill is another interesting direction for future work.

8 CONCLUSION

We presented continuous depth batching, the first inference method that serves depth-adaptive looped LMs end to end. By decomposing decode steps into separate stage queues, CDB turns the variable-depth forward pass into a scheduling problem that existing hardware can handle efficiently. A lookahead gate and asynchronous batch preparation hide nearly all per-step overhead, and a refill mechanism keeps the recurrent core’s batch size high. Our analytical framework (a FLOP-based speed-up bound and a roofline model) further shows when depth-adaptive decoding is beneficial and how the prelude-core-coda split reduces the speed-up. In our performance evaluation, CDB closely tracks the theoretical speed-up bound on Ouro 1.4B and Huginn 3.5B. These results demonstrate that looped LMs served with CDB are not only parameter-efficient but also inference-efficient.

REFERENCES

- Quentin Anthony, Jacob Hatf, Deepak Narayanan, Stella Biderman, Stas Bekman, Junqi Yin, Aamir Shafi, Hari Subramoni, and Dhabaleswar Panda. The case for co-designing model architectures with hardware. In *Proceedings of the 53rd International Conference on Parallel Processing, ICPP ’24*, pp. 84–96, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400717932. doi: 10.1145/3673038.3673136. URL <https://doi.org/10.1145/3673038.3673136>.
- Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise lora, 2025a. URL <https://arxiv.org/abs/2410.20672>.

- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=QuqsEIVWIG>.
- Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. In *8th ICML Workshop on Automated Machine Learning (AutoML)*, 2021. URL <https://openreview.net/forum?id=1EuxRTeOWN>.
- Song Bian, Tao Yu, Shivaram Venkataraman, and Youngsuk Park. Scaling laws meet model architecture: Toward inference-efficient LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=0TmVqOpBbK>.
- Michal Byra, Pawel Olszowiec, Grzegorz Stefanski, Grzegorz Gruszczynski, and Alberto Presta. bvit: Investigating single-block recurrence in vision transformers for image recognition, 2026. URL <https://arxiv.org/abs/2605.10661>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Markus Frey, Behzad Shomali, Ali Hamza Bashir, David Berghaus, Joachim Koehler, and Mehdi Ali. Adaptive loops and memory in transformers: Think harder or know more? In *Workshop on Latent & Implicit Thinking – Going Beyond CoT Reasoning*, 2026. URL <https://openreview.net/forum?id=F87X9c107e>.
- Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, and Yu Wang. Think-at-hard: Selective latent iterations to improve reasoning language models, 2025. URL <https://arxiv.org/abs/2511.08577>.
- Jonas Geiping, Sean Michael McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=S3GhJooWIC>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying

- Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, Sept 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, think, & generalize: Implicit reasoning in recurrent-depth transformers, 2026. URL <https://arxiv.org/abs/2604.07822>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP ’23, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Ryan Lee, Jacob Biloki, Edward J. Hu, and Jonathan May. Sparse layers are critical to scaling looped language models, 2026. URL <https://arxiv.org/abs/2605.09165>.
- He Li, Feichen Song, Boyi Zeng, Shixiang Song, Zhiqin John Xu, Ziwei He, and Zhouhan Lin. Ponderlm-3: Adaptive token-wise pondering with differentiable masking, 2026. URL <https://arxiv.org/abs/2603.02023>.
- Amirkeivan Mohtashami, Matteo Pagliardini, and Martin Jaggi. CoTFormer: A chain of thought driven architecture with budget-adaptive computation cost at inference. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=7igPXQFupX>.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024. URL <https://arxiv.org/abs/2404.02258>.
- Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How much is one recurrence worth? iso-depth scaling laws for looped language models, 2026. URL <https://arxiv.org/abs/2604.21106>.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=tVConYid20>.
- ShareGPT Team. ShareGPT, 2023. URL <https://sharegpt.com>.
- Wen-Jie Shu, Xuerui Qiu, Rui-Jie Zhu, Harold Haodong Chen, Yexin Liu, and Harry Yang. Loopvit: Scaling visual arc with looped transformers, 2026. URL <https://arxiv.org/abs/2602.02156>.
- Shixiang Song, He Li, Zitong Wang, Boyi Zeng, Feichen Song, Yixuan Wang, Zhiqin John Xu, Ziwei He, and Zhouhan Lin. Adaponderlm: Gated pondering language models with token-wise adaptive depth, 2026. URL <https://arxiv.org/abs/2603.01914>.
- Lintang Sutawika, Hailey Schoelkopf, Leo Gao, Baber Abbasi, Stella Biderman, Jonathan Tow, ben fattori, Charles Lovering, farzanehnakhaee70, Jason Phang, Anish Thite, Fazz, Aflah, Niklas, Thomas Wang, sdtblck, nopperl, gakada, researcher2, ttyuntian, Julen Etxaniz, James A. Michaelov, Chris, Chessing234, Hanwool Albert Lee, Janna, Leonid Sinev, Khalid, Kiersten Stokes, and Zdeněk Kasner. Eleutherai/lm-evaluation-harness: v0.4.12, May 2026. URL <https://doi.org/10.5281/zenodo.20122284>.
- Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. Sparse universal transformer. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://openreview.net/forum?id=yXYJPA1Lqn>.

- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following LLaMA model, 2023. URL https://github.com/tatsu-lab/stanford_alpaca.
- Yalcin Tur, Jalal Naghiyev, Haoquan Fang, Wei-Chuan Tsai, Jiafei Duan, Dieter Fox, and Ranjay Krishna. Recurrent-depth VLA: Implicit test-time compute scaling of vision-language-action models via latent iterative reasoning. In *Workshop on Latent & Implicit Thinking – Going Beyond CoT Reasoning*, 2026. URL <https://openreview.net/forum?id=hsIm52gD9p>.
- Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-efficient looped transformer: Decoupling compute from memory in looped language models, 2026. URL <https://arxiv.org/abs/2605.07721>.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, April 2009. ISSN 0001-0782. doi: 10.1145/1498765.1498785. URL <https://doi.org/10.1145/1498765.1498785>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL <https://aclanthology.org/2020.emnlp-demos.6/>.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-28-1. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models, 2025. URL <https://arxiv.org/abs/2510.25741>.

A RECURRENT-STEP LATENCY DETAILS

We measure decode latency of a single recurrent step on an 80 GB H100 GPU through our CB implementation with CUDA graphs and paged FlashAttention-3 (Kwon et al., 2023; Shah et al., 2024). For each batch size B and context length L , we submit B equal-length requests at full depth (no early exit). We set up the CB scheduler to complete prefill before any decode begins, so all B requests start decoding together from context length L and the decode batch size is exactly B .

For each (B, L) , we measure the wall-clock time τ of a full decode step by averaging over 16 consecutive token generations. We then separate the fixed boundary cost from the per-step cost by varying r . To isolate the recurrent-step latency, we vary $r \in \{1, 2, 3, 4\}$ for Ouro and $r \in \{2, 4, 8, 12, 16\}$ for Huginn and fit

$$\tau(r) = \tau_0 + \tau_r \cdot r, \quad (3)$$

where τ_r is the recurrent-step latency, and τ_0 is the prelude and coda overhead. For Ouro, $\tau_0 \approx 0.4$ ms across all operating points (embedding and LM head only). For Huginn, τ_0 ranges from 2.8 ms at short context ($L = 128$) to 4.6 ms at $L = 8192$ (due to transformer layers in prelude and coda), about $1.2\tau_r$. The fit is essentially perfect ($R^2 \geq 0.999$), confirming that decode latency is linear in depth.

We then fit the recurrent-step latency τ_r with the roofline model of Section 5, $t(N_c, N_b) = \delta + \gamma N_b + \alpha N_c$ (Williams et al., 2009). At a fixed context length L , each of the $N_b = B$ batch tokens reads L KV entries, so $N_c = BL$ and the model reduces to

$$t(B) = \delta + (\gamma + \alpha L) B. \quad (4)$$

The intercept δ is the context-independent weight reload cost, γB (mainly) the cost from matrix multiplications, and αLB the KV cache streaming cost. The saturation batch size is $B^* = \delta/(\gamma + \alpha L)$, which falls with L as the KV-cache streaming cost αL grows.

Fig. 7 shows the fitted curves including the larger Ouro model with 2.6B parameters. The weight-reload cost δ depends on layer shape (Anthony et al., 2024). Huginn’s recurrent core has 1.6B parameters in four wide layers, while Ouro 1.4B has 1.2B in 24 narrow layers. Huginn’s $\delta \approx 2.3$ ms is less than half of Ouro 1.4B’s $\delta \approx 5.3$ ms. Ouro 2.6B doubles the layer count to 48, which doubles δ to about 10.6 ms but leaves the slopes and therefore B^* essentially unchanged. For Ouro 1.4B, B^* drops from about 180 at $L = 128$ to single digits at $L = 8192$. Huginn follows a similar trajectory from about 80. Because the measured curves approach the compute-bound asymptote from below without fully reaching it at the largest batch that fits in memory, the fitted slopes mildly underestimate the true per-token cost, so these B^* values are mild overestimates.

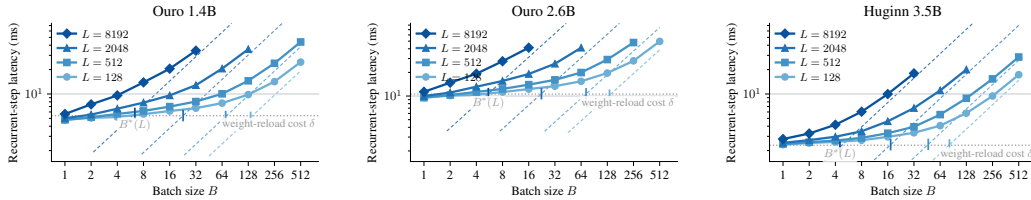


Figure 7: Decode latency of a recurrent step versus batch size at context lengths L , for Ouro 1.4B (left), Ouro 2.6B (center), and Huginn 3.5B (right). The dotted line shows the weight-reload cost δ , with blue ticks marking the saturation batch size $B^*(L)$. The dashed line shows compute-bound scaling, where latency grows linearly.

B BENCHMARK WORKLOADS

B.1 INPUT AND OUTPUT TOKENS

In our experiments, we do not let the models generate their own responses, as they may deviate between runs. Instead, we take input and output token lengths from the ShareGPT (ShareGPT Team, 2023) and Alpaca (Taori et al., 2023) datasets. From ShareGPT we take one request per conversation by using the first user-assistant pair, so each request is single-turn. From Alpaca we pair each

instruction with its reference output. We tokenize every prompt and reference output with the models’ tokenizer, drop pairs with fewer than four prompt or output tokens, and drop a few outlier requests whose combined length exceeds a context length of 16,384 tokens. This leaves 56,351 ShareGPT and 49,581 Alpaca requests. Because ShareGPT requests are an order of magnitude longer, we take a random subsample of 10,000 requests for it.

Fig. 8 shows the resulting input and output token length distributions under the Ouro tokenizer (Zhu et al., 2025). Alpaca requests are short and tightly concentrated (mean prompt and output lengths of 18 and 60 tokens), whereas ShareGPT requests are long and heavy-tailed (mean 270 and 310 tokens, with a 99th prompt-length percentile of 3264 tokens against Alpaca’s 75). Under the Ouro tokenizer, ShareGPT prompts and outputs are on average $15.0\times$ and $5.2\times$ longer than Alpaca’s.

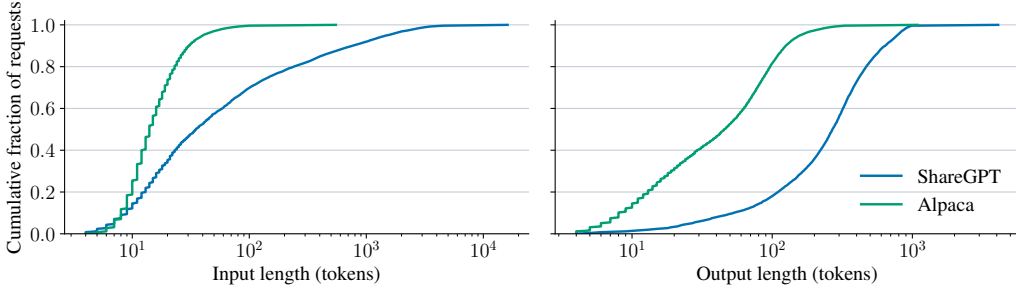


Figure 8: Empirical CDFs of prompt (left) and output (right) token lengths for the two workloads under the Ouro tokenizer. ShareGPT is long and heavy-tailed; Alpaca is short and tightly concentrated.

B.2 EARLY-EXIT DISTRIBUTION

Similar to the output tokens, we do not use the exit gate during our experiments. To make exit decisions deterministic, we run all requests through the models once in advance and record their per-token exits, which are then replayed during the experiments. Fig. 9 shows the resulting exit-depth distributions for different gate thresholds for Ouro and Huginn. For Ouro, a higher gate threshold makes fewer tokens exit early; for Huginn’s convergence gate, it is inverted. In both cases, ShareGPT and Alpaca produce nearly identical exit distributions despite their very different token length statistics. Huginn’s gate spreads exits over a wider range of depths.

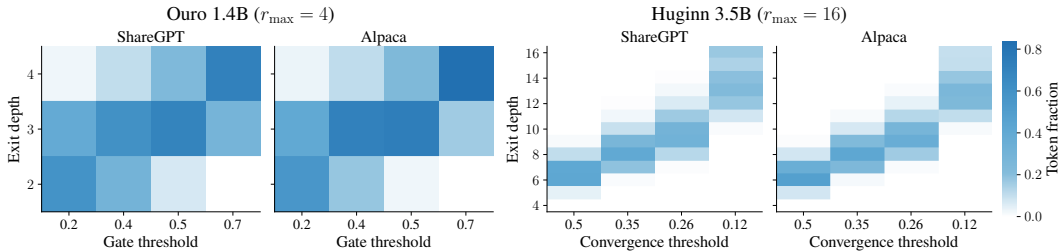


Figure 9: Exit-depth distributions for different gate thresholds. Each cell shows the fraction of tokens exiting at a given depth.

C ACCURACY-EFFICIENCY TRADE-OFF ABLATIONS

Our work is about maximizing the serving efficiency of looped LMs. However, this section evaluates how well current models support the techniques we build on. For this, we reimplement the `gsm8k_cot` task configuration of the LM Evaluation Harness (Cobbe et al., 2021; Sutawika et al., 2026) on top of our continuous-batching and continuous-depth-batching implementations, and evaluate Ouro 1.4B and Huginn 3.5B. Two choices in our setup affect task accuracy, the KV cache policy and the exit gate; both trading accuracy for throughput and memory.

C.1 KV CACHE SHARING

We test how KV cache sharing affects accuracy and memory, evaluating both models at fixed depth. We compare a full depth-indexed cache against two shared layouts: single slot, where every loop overwrites one KV slot per physical layer, and first-then-shared, where the first loop step keeps a private slot and all later ones share a second. Neither model was trained for this setting, so KV sharing is a zero-shot capability here.

The results in Table 1 show that Huginn is insensitive to the KV sharing layout, while Ouro loses accuracy under both layouts, from about 6 points under first-then-shared to a complete collapse under single-slot sharing. We cannot reproduce the results from the Ouro paper, which reports near-lossless KV sharing (Zhu et al., 2025). This might be because they share only during decoding and keep a depth-indexed cache for the prompt. Still, they report a degradation of about 10 accuracy points once the sharing extends to prefill, which is far less than the drop we observe. Our results are in line with Vendrell et al. (2026), who propose integrating KV sharing into Ouro’s training, which then recovers most of the gap. Furthermore, Huginn’s insensitivity holds only at full depth. Surprisingly, sharing a single KV slot under a truncated loop (i.e., reducing $r_{\max}$) retains substantially more accuracy than the full depth-indexed cache: 10.99% against 3.87% at fixed depth 6 and 25.32% against 11.75% at depth 9, converging to within about a point at depth 16.

Overall, sensitivity to KV cache sharing depends on the model, and decreasing it should be a development target, as KV sharing enables not only much simpler serving but also substantial memory savings.

Table 1: GSM8K accuracy (flexible match, greedy decoding) and KV cache footprint under different recurrent-depth KV cache layouts. KV slots are counted per physical layer and token. Cache sizes assume `bfloat16` and are reported per token and for one full 4096-token sequence.

KV cache layout	OURO ($r = 4$)				HUGINN ($r = 32$)			
	Slots	KiB/tok	GiB@4k	Accuracy	Slots	KiB/tok	GiB@4k	Accuracy
Full depth-indexed	4	768	3.00	77.86%	32	2722	10.64	33.74%
Single shared	1	192	0.75	0.23%	1	165	0.65	33.97%
First-then-shared	2	384	1.50	71.34%	2	248	0.97	34.80%

C.2 EARLY-EXIT GATING

We evaluate how depth-adaptive decoding affects accuracy compared with fixed-depth decoding. We are also interested in the effect of the delayed exit decision of our asynchronous CDB scheduler (Section 4.3). For Ouro we take the published gate for the baseline comparison and train two additional exit heads on the frozen model (via gate distillation) to naturally support our serving pipeline: (1) a lookahead gate that predicts the exit one step in advance and (2) a pre-loop gate that predicts the exit before the loop starts. Note that we set $r_{\min} = 2$ because Ouro is not trained to make useful predictions after the first loop, which would then drag down accuracy if the model exits at this step (Zhu et al., 2025). This also has the side effect of testing our serving setup, where the delayed exit imposes $r_{\min} = 2$. Huginn has no internal gate, so we use the training-free convergence criterion of Geiping et al. (2025), which exits once the L2 distance between consecutive recurrent states falls below a threshold. For delayed exits we use the same criterion. We evaluate Huginn with $r_{\max} = 16$ recursions rather than its trained 32, since we found fixed-depth accuracy to be flat within noise above 16. We also use the shared KV cache layout for better performance (Appendix C.1).

Fig. 10 summarizes the results. The gated models Pareto-dominate fixed exits, reaching higher accuracy with lower mean loop depth. Ouro performs well down to a mean depth of ≈ 2.5 and Huginn down to ≈ 8 . The exit distributions in the right panel show that Ouro spreads tokens over three of its four depths and Huginn over four to eight of its sixteen.

Regarding delayed exits, we observe that the trained lookahead gate matches the exit behavior of the teacher gate very well and leads to approximately the same accuracy and threshold curves. Thus, the exit can be predicted one step earlier just as well as in the same step, validating our scheduler. Knowing the exit decision before the loop starts would make the loop exits more predictable, offering

the scheduler even more opportunities to accelerate serving. However, the pre-loop gate does not perform well, confirming the observation of [Bae et al. \(2025b\)](#). For Huginn, the delayed execution of the exit decision comes for free and only requires a threshold recalibration.

Overall, we observe that gating is a good technique, and our delayed exits in scheduling do not affect accuracy in these models. However, the minimum recursion depth remains 2 for Ouro, and 3 for the training-free convergence criterion (it compares two consecutive latent states, and the delayed scheduler adds a further step), limiting the exit distribution and the amount of compute that can be saved. Ouro and Huginn naturally fit our setup, but shallower looped LMs might be affected more.

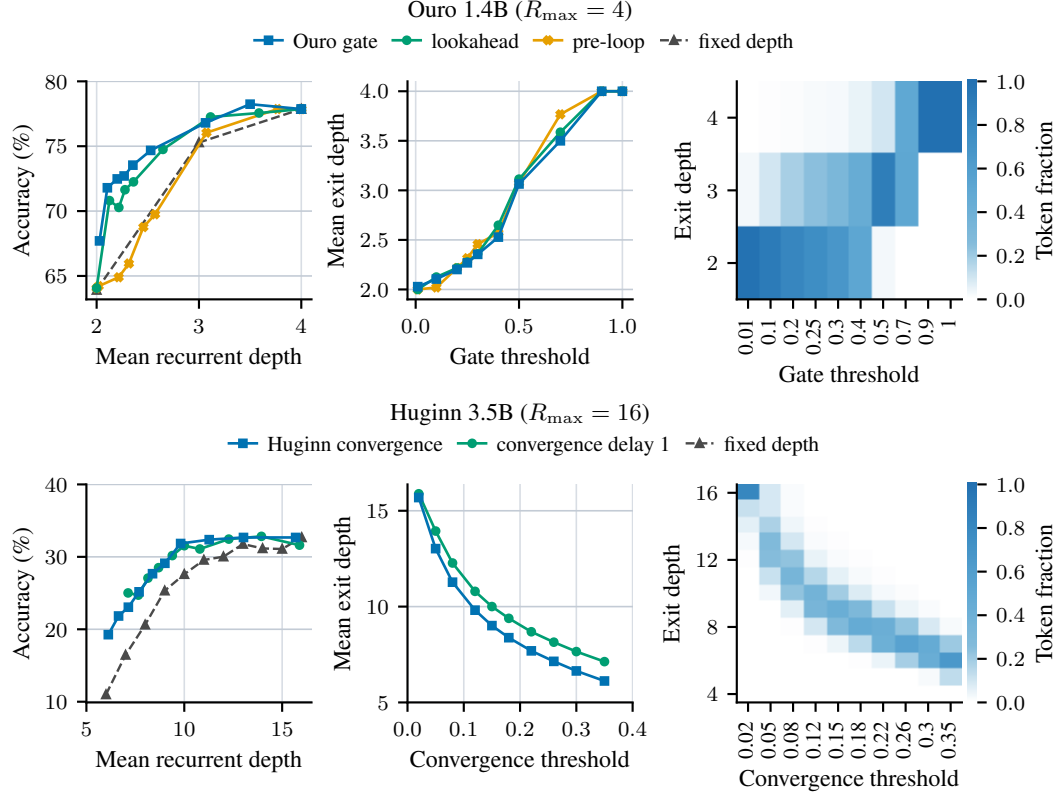


Figure 10: Depth-adaptive decoding against fixed-depth decoding on GSM8K (flexible match, greedy decoding), for Ouro (top) and Huginn (bottom). Each row shows the accuracy–depth frontier (left), the mean exit depth for each threshold (middle), and the exit-depth distribution (right). The distribution panel shows each model’s own exit policy (“Ouro gate” and “Huginn convergence”).